

Chapter 3

Security Foundations

THE FOLLOWING CEH EXAM TOPICS ARE COVERED IN THIS CHAPTER:

- ✓ Elements of information security
 - ✓ Risk management
 - ✓ Information assurance
-

Organizations generally spend a lot of time and money on defenses and mitigations against attacks. There are some fundamental concepts that go into the planning and implementation of these defenses. In this chapter, we're going to cover some of the subject matter that helps security professionals make decisions about how best to protect enterprises. Some of this is foundational, in the sense that you may not find it specifically on the CEH exam, but it's necessary to cover to be able to build on it. By the end of the chapter, you'll have the basics behind you, and we'll have started to talk about hard, defensive mechanisms. You will run across many of these if you are acting as an ethical hacker.

First, you need to understand what is meant by information security—what events fall into the security bucket. These ideas are commonly referred to as the triad or the CIA triad. It's an essential concept for people who hold a Certified Information Systems Security Professional (CISSP) certification, which is a certification that covers the entire gamut of information security topics. The CIA triad is a good foundation to understanding so much else about information security.

Alongside knowing what you are protecting against, you need to know *what* you are protecting. Performing risk assessments will help with that. Identifying potential risks and associated potential losses can help with decision making around where to expend limited resources—budget, staffing, capital expenditures, and so on. It will also help security professionals determine what policies they need in place to guide the actions of employees overall and, more specifically, information technology staff. From policies flow standards and procedures. They are all considered essential for determining how a company approaches protecting its information assets.

Once you have your policies in place, decisions can be made about how best to protect information resources. Beyond staffing, there is technology that can be brought to bear. There are a number of devices that can be placed into a network to help with information protection. These are meant not only to keep attackers out but also to provide information that can be used to identify intrusions, including the path an attacker may have taken to gain access to networks and systems. There may also be elements that are placed in the network that can automatically detect an intrusion attempt and then block that attempt.

The technology alone doesn't provide protection. Where these elements and devices are placed is important, and those decisions are made based on ideas such as defense in depth and defense in breadth. These ideas help to define a complete security architecture. Alongside the security devices, there are system-level mechanisms that can be very helpful in providing more context to the information provided by firewalls and intrusion detection systems. All of these elements and decisions together are needed to create a complete defense of the enterprise.

The Triad

The triad is a set of three attributes, or properties, that define what security is. The three elements are confidentiality, integrity, and availability. Each of these properties needs to be taken into consideration when developing security plans. Each of them, to varying degrees of importance, will be essential in planning defenses. Not every defense will incorporate each of the three properties to the same extent. Some defenses impact only one property, for instance. From an attack perspective, a tester or adversary would be looking to compromise one of these elements within an organization. You will commonly see these three elements referred to as the CIA triad. This is not intended to be confused in any way with the Central Intelligence Agency.

Figure 3.1 shows the three elements represented as a triangle, as the triad is often depicted. One reason it is shown that way is that with an equilateral triangle, all the sides are the same length. They have the same value or weight, which is meant to demonstrate that none of the properties of the triad is any more important than any of the other properties. They all have equal value when it comes to the overall security posture of an organization. Different situations will highlight different elements, but when it comes to the overall security of an organization, each is considered to have equal weight to have a well-balanced approach.

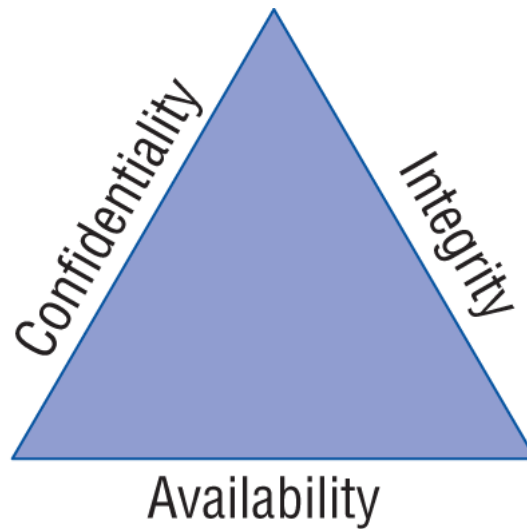


FIGURE 3.1 The CIA triad

Something else that might occur to you, as you look at the figure, is that if any of the sides are removed or compromised, it's no longer a triangle. The same is true when it comes to information security. If any of these properties is removed or compromised, the security of your organization has also been compromised. It takes all of these properties to ensure that you and your information are being protected.

Confidentiality

Perhaps we should go back to kindergarten for this, though you may have more recent memories or experiences that apply. There are always secrets when you're young, and it seems like they are always shared in very conspiratorial, whispered tones on the very edge of the schoolyard. You expect that when you share a secret with your friend, that secret will remain between you and your friend. Imagine, after all, if you were to tell your friend that you really, really, really liked the person who sat behind you in class and your friend were to tell that person. That would be mortifying, though the mortification level would likely be in inverse relationship to your age.

When your friend shares your secret, assuming they do, they have breached your confidence. The secret you have shared with them is no longer confidential. You won't know whether you can continue to trust your friend. One of the challenges is that you might have shared your secret with two friends. You then hear about the secret from someone else altogether. You don't know without probing further which friend may have been the one to tell. All you know is that confidentiality has been breached.

In the digital world, confidentiality still means keeping secrets, so nothing much changes there. This, though, encompasses a lot of different aspects. It means making sure no one gets unauthorized access to information. This may mean using strong passwords on your user accounts to make sure attackers can't get in. It may mean keeping certain information offline so it can't be accessed remotely. Commonly, though, a simple way to achieve confidentiality is through the use of encryption.

When we are talking about confidentiality here, we should be thinking about it in two dimensions—static and dynamic. Static would be protecting data that is considered “at rest,” which means it's not moving. It is probably stored on disk and not being used or manipulated. The second type, dynamic, is when the data is moving, or “in motion.” This refers to data when it is being sent from one place to another. This may include your web browser asking for and then retrieving information from a web server. As the data is being transmitted, it is in motion. It's this transmission that makes it dynamic and not necessarily that it is being altered, though data being sent from one place to another could definitely be experiencing alteration through interaction with the user and the application.

When we are making use of encryption for web-based communication, the Secure Sockets Layer/Transport Layer Security (SSL/TLS) protocols are used. While TLS has long since superseded SSL, it is still sometimes referred to as SSL/TLS. Regardless of how it's referred to, though, it is a set of mechanisms for encrypting data. SSL and TLS both specify how to generate encryption keys from data that is known, as well as some partial data that is transmitted from one side to the other.

Since encrypted data can't be read without a key, the confidentiality of the data is protected. This is not to say that encryption guarantees confidentiality. If an attacker can get to the key in some way, the attacker can then decrypt the data. If the data is decrypted by someone who shouldn't have seen it, confidentiality has of course been compromised. Attacks against the encryption mechanisms—ciphers, key exchange algorithms, and so on—can also lead to a compromise of confidentiality. A successful attack against the encryption—and there have been successful attacks against various encryption methods and standards—will lead to ciphertext being decrypted.

Integrity

In addition to having data be confidential, we also generally expect it to be the same from the moment we send it to the moment it's received. Additionally, if we store the data, we expect the same data to be intact when we retrieve it. This is a concept called *integrity*. Data integrity is important. It can be compromised in different ways. First, data can be corrupted. It can be corrupted in transit. It can be corrupted on disk or in memory. There are all sorts of reasons data gets corrupted. Many years ago, I started getting a lot of corrupted data on my disk. I replaced the disk and still got a lot of corrupted data. In the end, it turned out that I had bad memory. The bad memory was causing data being written out to disk to be corrupted. Bad components in a computer system happen, and those bad components can result in data loss or corruption.

Sometimes, mistakes happen as well. Perhaps you have two documents up and you are working in them at the same time. One of them is a scratch document with notes for the real document. You mistakenly overwrite a section of the real document, thinking you are in the scratch document, and then, because you don't trust autosave, you save the document you are working in. This might be considered a loss of data integrity be-

cause important pieces of the document you are working in have been altered. If you don't have a backup from which to replace data, you may not be able to recover the original, and it could be that you don't even notice the mistake until the document has been sent around the office with the text containing the mistake in it. Suddenly, information is being shared that is incorrect in some way.

A man-in-the-middle attack is one way for an attacker to compromise integrity. The attacker intercepts the data in transit, alters it, and sends it on the way. When you browse to a website that uses encryption, a certificate is used to provide keying, meaning the certificate holds keys used for encryption and decryption. When a certificate contains a name that is different from the hostname being visited, your browser will generate an error, as you can see in [Figure 3.2](#).

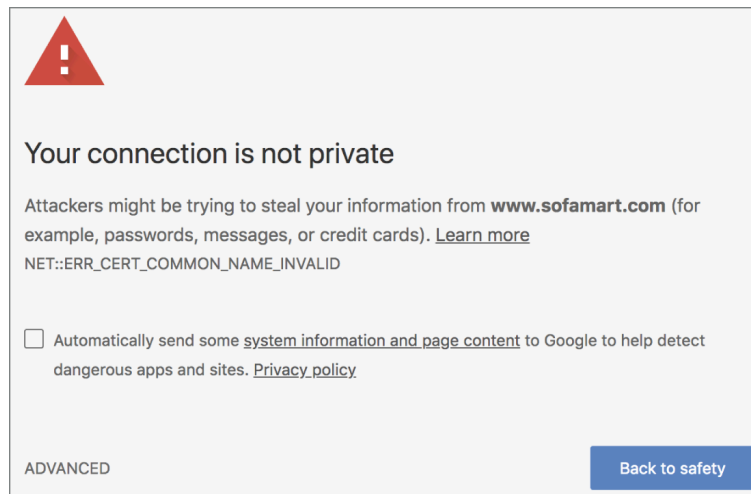


FIGURE 3.2 An error message about an apparently invalid certificate

What the certificate says, though it's not shown in the error, is that it belongs to [www.furniturerow.com](#). However, the website that was visited was [www.sofamart.com](#). If you didn't know that the Furniture Row company owned Sofa Mart, you might begin to question whether you were being hijacked in some way. An attack in this situation may have resulted in an attacker gathering information from your session, which you wouldn't have expected because you believed the session was encrypted. Worse, the information you got could have been altered while the attacker got the real information. There are many cases where that may be a realistic scenario.

Integrity, it turns out, is very complex. There are so many cases where integrity can be violated and we've only scratched the surface. Integrity isn't only about the contents of the data. It may also be the integrity of the source of the information. Imagine writing a letter and digitally adding a signature so it appears to have been written by someone else. When you start thinking about integrity, you may come up with many examples.

Availability

This is perhaps the easiest to understand and one of the most commonly compromised properties. It comes down to whether information or ser-

vices are available to the user when they are expected to be. As with the other properties, this may be a case of mistakes and not necessarily malicious. If you were to keep information on an external drive, then go somewhere—work, on-site with a client—and forget to bring the drive, the files on that drive wouldn't be available. It wouldn't be malicious, but it would still be a failure of availability since you wouldn't be able to access what you need when you need it. You haven't lost the data. It's still intact. It just isn't where you need it when you need it to be there. That's a breach of availability.

Misconfigurations can result in problems of availability. Making a change to a service configuration without testing the change may result in the service not coming back up. In production, this can be very problematic, especially if it's not clear that the service failed. Some services will appear to have restarted when in fact the configuration caused the service start to fail. Even if it's a short period of time before the change has been rolled back, there is a possibility of users of the service not being able to perform the functions they expect.

Recently, I was reading about a case of a cluster where six out of the seven devices took an upgrade cleanly while the seventh device was still on the older code. Because there were some incompatibilities with the communication between the older code and the newer code, all the devices went into a bad loop, shutting out all legitimate requests.

Malicious attacks are common as well. A denial-of-service (DoS) attack denies access to a service, which translates to the service being unavailable for legitimate traffic. Attackers can overwhelm the service with a lot of requests, making it difficult for the service to respond. A legitimate request would have a hard time getting through the noise, and even if the request got through, the service may not be able to respond. DoS attacks have been common for decades now, though the change in available bandwidth and other technology changes means that the attacks have had to change along with the technology and bandwidth increases.

Parkerian Hexad

Not everyone believes that three properties are sufficient to encompass the breadth of information security. In 1998, Donn Parker extended the initial three properties by adding three more. These are not considered standard, because there is some debate as to whether it's necessary to break the additional properties out. The three additional properties Parker believes are necessary are as follows:

- **Possession (or Control)** If you had mistakenly handed the external drive mentioned earlier to a friend, thinking you were handing them back their drive, the drive would be in their control. If the friend never plugged the drive in to look at it, the data on it would not be subject to a breach of confidentiality. However, the fact that the drive is not in your control any longer does mean that it is not available to you, meaning it is a loss of availability. This is one reason this property isn't included in the primary triad, though it can be an important distinction.

- **Authenticity** This is sometimes referred to as nonrepudiation. The idea of authenticity is that the source of the data or document is what it purports to be. As an example, when you digitally sign an email message, the recipient can be certain that the message originated from you because the message includes your digital signature, which no one else should have. Of course, in reality, all we know is that the message was signed with your key. If your key was stolen or given away, it could be used without you. Authenticity is making sure that when you get a piece of data, no matter what it is, it's actually from where it purports to be from.
- **Utility** Let's say you have that same external drive we've been talking about. It's now many years later, and it's been in a drawer for a long time. It's just the drive, though. There is no cable with it because they got separated. You don't have the cable anymore, and additionally, the interfaces on your computer have all changed since the last time you used it. Now you have data, but it cannot be used. It has no utility. Personally, I have an old nine-track tape sitting in a tub in the basement. I've had it for decades. It's completely useless to me because I don't have a tape drive or a mainframe to read it to. Additionally, it's likely stored in EBCDIC and not ASCII. All of this makes the data on that tape useless, despite that it is in my possession and technically available to me.

While these properties do raise specific ideas that are important to think about, you could also fit the scenarios into the three properties that are part of the CIA triad. Possession and utility situations could be said to fall under availability, and authenticity could be placed under integrity. After all, if the source of a piece of data is said to be one thing when in fact it isn't, the integrity of the data is invalid. What you get with the Parkerian hexad is some specific cases rather than the broader concepts from the CIA triad. If it's more useful for you to break them out from the others as you are thinking about what security means and what events are security-related, the hexad may be more useful for you than the triad.

Information Assurance and Risk

Very simply, risk is the intersection of loss and probability. This is a condensed idea, and it can take a lot to unpack, especially given common misunderstandings of what risk is. A longer version of this sentiment is found in the definition of *risk* at [Dictionary.com](https://www.dictionary.com), which says that risk is “the exposure to chance of injury or loss.” The chance in the definition is the probability, which is measurable. Loss or injury is also measurable. This means we can apply numbers to risk and it doesn't have to be something amorphous or vague.

Often, you will see the term *risk* used when what the speaker really means to say is *chance*, or *probability*. Someone may say there is a risk of some event happening. What is really meant is that there is a probability of that event happening, and presumably, the event is one that could be perceived as negative. Probabilities can be calculated if you have enough data. A very simple way to calculate probability, which is commonly ex-

pressed as a ratio, is to divide the number of events by the number of outcomes.

As an example, what is the probability of any day in April falling on a weekend? There are 30 days in April. That's the number of outcomes. As there are typically 8 weekend days in a 30-day month, the number of events is 8. The probability then is 8/30, or 8 out of 30. If you wanted to, you could reduce that to 4 out of 15, but 8 out of 30 says the same thing, and it's clearer to see where the information came from. If you wanted to refine that, you could ask about a specific April to see if, based on how the days aligned, there were more than 8 weekend days that year.

Probabilities of information security events are harder to calculate. What is the probability of your enterprise being hit by a distributed (based on the acronym you are using) denial-of-service attack? According to Imperva Incapsula, using its online DDoS Downtime Cost Calculator, the probability of a 2,500-person company that is in the e-commerce business getting hit with a DDoS is 36 percent. That's 36 events out of 100 outcomes. In this context, we start getting very amorphous. What is an event here? What is an outcome? For every 100 connection attempts to your network, 36 will be DDoS messages? That doesn't make sense. As I said, calculating the probability of risk is challenging, and even if you do, it may be hard to understand the results you get.

Loss is, perhaps, easier to quantify, though even there you may run into challenges. Let's say that your business has been compromised and some of your intellectual property has been exfiltrated. Since the intellectual property isn't gone, meaning you still have it in your control, what is the tangible loss? It depends on who took it and what they took it for. If someone in another country took it and your business doesn't sell any product there, is there an actual loss?

Certainly, there are costs associated with cleanup. Even there, though, it can be challenging. So much is soft costs. If you don't bring in an outside contractor to do the cleanup for you, you are using your own people, whom you are already paying. An outside contractor will have hard costs in the form of a bill your company will be expected to pay. That will come out of your bottom line, so there is, for sure, a number that can be applied to loss there. How do you calculate the cost of cleanup if it's your own people, though? You're already paying them, so the cost is in deferred actions on other projects.

Now, we know we can get values for loss and for probability. We can calculate risk from those two values by multiplying loss by probability. You end up with $\$risk = probability * loss$. The dollar sign is included in there to be clear about the terms so you know what the end result means. The risk value you end up with is in dollars. This value is more meaningful in a comparative way. You can compare the risk of different events by using quantitative comparison if you know the monetary value of the loss and the probability. This means you are not comparing just loss and not just probability.

It can be tempting to think about risk as loss. A high-risk event to some is one where there is a catastrophic outcome. This is the way our brains are wired. Our brains look for bad things and try to avoid them. Because of that, if you were to ask someone what a high-risk event in their lives is, outside of information security, they may well give you an example of an event that may result in death, like driving. They don't take into account the actual probability of a catastrophic event happening. They only think about the potential outcome at its most extreme. This is called *catastrophizing*. It's not helpful when we are trying to evaluate risk in an information security context.

Risk is used as a way of identifying what is important. The goal of any information security program is to protect what is important and valuable. One way to quantify what is valuable is through the loss number. Higher loss may mean higher value. When you calculate the overall risk value, you can determine where to apply your resources. If you have an event that has a high risk value, it is probably a good idea to apply resources to protect against that event. Of course, predicting every possible event can be challenging. This is what makes information security difficult—understanding events, probabilities, and outcomes for your organization and then planning to protect against negative events.

When it comes to risk, there are other concepts that are important to consider. The first is threat. A threat is something that has the possibility to incur a breach of confidentiality, integrity, or availability. The avenue this breach may take is called a vulnerability. A vulnerability is a weakness in a system. This may be its software, its configuration, or how the entire information solution is put together. When the vulnerability is triggered, it is called an exploit. We exploit vulnerabilities, though not all vulnerabilities can be exploited. Race conditions are examples of vulnerabilities that may not be able to be exploited. A code review shows that there is a problem and the result of the problem could result in data corruption, for example. However, because of the speed at which the program executes, it's essentially impossible to sit in between the two instructions to make the change. Certainly not all vulnerabilities can be exploited by everyone.



A race condition is a programmatic situation where one process or thread is writing data while another process or thread is reading that data. If they are not tightly in sync, it's possible for the data to be read before it's written. It may also be possible to manipulate the data in between writing and reading. At its core, a race condition is a synchronization problem.

A threat agent or threat actor is an entity, like a person or group, that can instantiate a threat. The threat agent is who manifests a threat. The pathway the threat agent takes to exploit a vulnerability is called the threat vector. All of these concepts are important because they help to better understand where risk lies. Once you have identified what resources you

care about, you should think about what threat agents may care about those resources. This can help you to identify potential vulnerabilities. Once you know what your vulnerabilities are and the potential threat vectors, you can start to think about how you are going to protect against those threats.

Information assurance is the practice of identifying risks associated with information assets, then instituting controls to better protect the asset or resource. A control is a means of ensuring the confidentiality, integrity, or availability of an information security resource. These controls may be preventive, detective, or corrective. These are the goals or functions of the controls. A preventive control is one that intends to keep something bad from happening. Unfortunately, this is not always possible, which means detective controls are useful. A detective control is one where a violation of confidentiality, integrity, or availability is identified so the violation can be addressed. Finally, a corrective control is one that intends to limit the amount of damage that may have been caused.

Security controls end up having two factors. In addition to controls having goals, as described earlier, they will fall into one of three different types. A control may be technical, administrative, or physical. A technical control is hardware or software that has been implemented to provide protection of assets. This may be an authentication solution, a firewall, antivirus, or a security incident and event manager used to alert on something bad happening. An administrative control is a policy, procedure, or guideline that gives direction on how protections should be implemented. Finally, physical controls are used to keep people away from assets in a kinetic way, meaning someone can't walk up to a building and just enter it. A physical control doesn't talk about logical access, such as logging into an SSH server remotely. A physical control may be a door lock or it may be a closed-circuit television camera used to observe activities to detect unwanted behaviors.

Information assurance is about managing risk. This means not just identifying risk and how to categorize it, but also making business decisions about how to handle the risk. Businesses may commonly handle risk in different ways, depending on the level of risk and their risk appetite, which means the level of risk they feel comfortable managing. These are the ways risk is commonly handled within businesses.

- **Acceptance** Risk acceptance is an informed process where the business fully understands the nature of the risk, meaning the impact (the amount of money the actualized risk will incur) and probability, and chooses not to do anything about the risk. This means if the risk is actualized, the business will be responsible for all the losses and costs associated with it, as well as any other damage that may result. This approach means nothing else is done to the circumstances related to the risk identified.
- **Transference** The business may choose to transfer the risk to another entity to manage in some way. One common way to implement risk transference is to buy insurance. You very likely have auto and home insurance for your car and house (or maybe rental insurance if you are renting). You have chosen to transfer risks associated with

those assets to the insurance company. This means an exchange in the case of insurance—you can transfer the risk to them and they will take on any liabilities in exchange for money in the form of a premium.

- **Mitigation** Risk mitigation is implementing controls that will reduce the risk. Risk reduction can occur on the two vectors that constitute risk—loss and probability. You can implement a control that reduces the probability of a risk being actualized. This may be through implementing different technical controls to, for instance, restrict access to a resource to a limited number of people from specific locations.
- **Avoidance** Business may choose to not engage in the activity that could introduce risk to begin with. This is called risk avoidance. This is a legitimate practice, though if an activity is essential to the business, risk avoidance may not be possible. The business will need to choose another approach if they intend to get the benefit from the activity being proposed.

Policies, Standards, and Procedures

While we, as information security professionals, can often think about security for the sake of security as the most important thing, the fact is that security is a business enabler, not a business driver. This means security, in most cases, doesn't add to the bottom line. The lack of security or a weakness in security can take away from the bottom line. Because security is a business enabler, the business sets the parameters around what is important and the means to protect what is important. It does that by creating policies. Once the policies are created, standards are built out of those policies. Closest to where the work actually gets done are procedures. These are developed because of what the standards say.

Security Policies

A security policy is a statement of intention with regard to the resources of a business. It defines what a company considers to be security—what resources need to be protected, how resources should be utilized in a proper manner, how resources can or should be accessed. These policies are essential to good corporate governance, since the policies are lines that management draws. This means that having management set the tone and direction is not only a good idea, it's also required. Management and the board of directors have an obligation to the stakeholders in the business—the individual owners or the shareholders.

Security policies are not only about defining important resources, they are also about setting expectations of employees. Many organizations, for example, have an acceptable use policy. This is about defining what users can and cannot do. Any violation of this policy on the part of the employee is generally cause for sanction or termination, depending on the extent of the violation and the impact to the business. Of course, not all policies are directly about the users and their behaviors. There may be other security policies that are more directed at information technology or information security staff.

Keep in mind as you are thinking about security policy that the goals of your policies should be the confidentiality, integrity, and availability of information resources. These are the properties that a security policy should take into account. All information resources should be confidential, have integrity, and be available within the parameters defined by the business. This doesn't mean that all information resources should always be confidential, have integrity, and be available. Different information assets will have different levels of confidentiality, and not all information has to be available all the time. This is also part of what security policy is for—to help classify and prioritize the information assets of the organization.

Not all information resources are bits and bytes stored on computer systems. One area that policy should always take into consideration is how the human resources are to be handled. This may not always be codified in information security policy, but human resources should always be a factor, especially when it comes to protecting against natural disasters and other events that may be physically dangerous as well as impacting information assets.

Policies should be revisited on a regular basis. They are sufficiently high-level that they shouldn't change every time there is a new set of technologies available, but they should evolve with changes in the threat landscape. As information resources change and threat agents change, the policies may need to adapt to be responsive to those changes. As this is an area of corporate governance, any policy change would have to be approved by whatever management structure is in place—business owners, board of directors, and so on.

Keep in mind that security policies are all high-level. They don't provide specifics, such as how the policies should be implemented. If you're looking at security policies and you're starting to think about the operational impacts and how the administrator would handle the policy, you're too close to the ground, and it's time to beat your wings a bit more to get much higher up. Also, you are thinking conceptually for what should be long-term rather than something specific to a point in time. This means that technology and solutions have no place in the policy. Other information security program elements will take care of those aspects.

Security Standards

The security policy is at the top of the food chain. There may also be sub-policies that flow down from the top-level security policies. The subpolicy should refer to the overall policy so the high-level policy doesn't get lost. Below the policy level, though, are security standards. A standard is direction about how policies should be implemented. The standard starts down the path of how we get from statements of intent to implementation, so we start to drill down from the high level of the policy.

SECURITY STANDARDS

There are two meanings for the term *security standard*.

There are sets of standards that provide guidance for organizations and are managed by standards bodies. The National Institute of Standards and Technology (NIST) has a set of standards, documented in several special publications. The International Organization for Standardization (ISO) maintains ISO 27001 and ISO 27002. There are other standards documents that may be relevant to you, depending on where you are in the world.

The second type of standard is one that provides detailed guidance for how to implement policies within organizations.

Take, for example, a policy that states that all systems will be kept up-to-date. To get closer to implementation of that policy, you might have standards that relate to desktop systems, server systems, network devices, and any embedded devices. The requirements for each of those device types may be different, so the standards for them may be different. Desktop systems may just be expected to take all updates as they come, with the expectation that any potential outage could be remediated quickly on the off chance that there was an outage on a handful of users' desktops.

Servers, on the other hand, would be in place to service customers and possibly have revenue impacts. Since that's the case, the standard may be different. The standard, still focused on how to achieve the objective set out in the policy, may say that there is a quality assurance process that is necessary before patches may be deployed. The service level agreement (SLA) on those server systems may be completely different in terms of acceptable outages. The standard for the server systems may be different from the desktop systems, but both are written in service of the high-level policy that systems are kept up-to-date. The standard would define anything that was vague (what does "up-to-date" mean?) in the policy to make it relevant to operational staff.

The standards are still high level in the sense of setting requirements for how policies should be implemented. These requirements still need to be implemented. That leads us to another step.

Procedures

Procedures are the actual implementation of the standard. These provide guidance about how, specifically, the standards are achieved at a granular level. This may be accomplished with step-by-step instructions on what needs to be done. There may be multiple procedures for each standard, since there may be multiple organizations involved in implementing the standard.

You can see that with high-level guidance like that in a policy, you likely wouldn't have to touch it often. Policies are revisited on a regular basis,

but the time scale for when the policies change would be measured in years, if the policies are well considered and well written. Standards, though, may need to be updated more regularly. Information asset changes would result in standards being updated. Any change in technology within the organization may result in an update to standards. Procedures will likely change even more regularly.

As organizations in the company shift or responsibilities shift, procedures will shift to accommodate them. Additionally, a good procedure would have feedback loops built in so the procedure could be regularly revised to be more efficient. Any change in automation would result in procedure changes. As we go through each layer from the top of the security program down to the very bottom, the result is more regular updates as we get to specific steps in implementation and administration.

Guidelines

You may not run into guidelines as you are looking at security programs. Guidelines are not standards in that they may not be requirements. Instead, they are suggestions on how policies may be implemented. A guideline may provide information about best practices, with the hope that the best practices may follow.

Organizing Your Protections

There are a lot of different ways of thinking about information security, of course. From a governance perspective, you need to think about policies, standards, and procedures. However, once you have all of those in place, you still need to think about how you are going to best protect your assets, not to mention ensure you are able to detect any attempt to negatively impact those assets, no matter what they are. One way to think about evaluating protections of your assets is by reviewing what attackers are most likely to do. If you know what actions the attackers are going to take, you could develop not only protections but also detections of those actions, which helps you to be prepared. While you can use the Lockheed Martin cyber kill chain or the attack lifecycle advocated by FireEye Mandiant, another way to get even more detail is to make use of the MITRE ATT&CK Framework.

The MITRE Corporation developed ATT&CK in 2013 to collect common tactics, techniques, and procedures (TTPs) used by attackers. The way MITRE saw it, existing frameworks like the cyber kill chain were too high level and not applicable to real-world incidents. The idea behind ATT&CK is to collect these TTPs from adversaries, so the collection is always growing as more TTPs are identified. These TTPs are described in detail, so protection and detection controls can be developed.

While there are different attacker types, including hacktivists who may be just looking to make a point in the public consciousness, the most dangerous ones are ones that are sometimes called *advanced persistent threats* (APTs). These are attackers who use a large number of TTPs to gain access to a business environment and remain within the environ-

ment for as long as they can. In some cases, these attackers may remain in place for years. There have been cases in recent memory where attackers have been found to have remained in place for seven or ten years or more. While you may get some attackers who might do a one-and-done approach, where they are targeting individual users and want to just get what they can and get out, increasingly, what we see is these APT attackers. This is worth keeping in mind as you look through the different phases of the ATT&CK framework.

The ATT&CK Framework is broken out into the following stages. You may find these aren't necessarily in the order an attacker may follow, as there may be some overlap or even bouncing back and forth. Additionally, some TTPs will fit into different categories here.

- **Reconnaissance** The attacker is gathering information about the target in this stage. They may be searching through open sources where data is freely available. They may also be using closed sources they have access to. Additionally, they may be performing scanning activities here. This stage also includes attackers using phishing techniques to trick victims to give up data.
- **Resource Development** The attacker has to do a lot of work to create an infrastructure they will use to launch attacks from. This may include registering domain names, creating accounts, acquiring systems (probably by compromising other victims and using their computing infrastructure). They may also need to acquire tools they don't have themselves. This could be renting or purchasing toolset that may be available through criminal channels.
- **Initial Access** The attacker here compromises the first systems. They may compromise an existing public-facing application or website. They may use phishing to get malware installed on a target system. They are getting their initial foothold into the organization in this stage. Any technique that gives them access, even if it's as simple as making use of a compromised set of credentials, falls into this stage.
- **Execution** There are a lot of techniques an attacker may use to get code executed. The attacker doesn't always have direct interactive access to the system, meaning they can't just double-click an icon or run a command-line program. They may rely on other techniques to get their code executed. There are a lot of ways of doing this, including using scheduled tasks, interprocess communication, system services, or operating system-specific techniques like using the Windows Management Instrumentation (WMI) system.
- **Persistence** With persistence, the attacker is trying to maintain consistent access to the system they have compromised. Systems reboot, sometimes they are powered down and later restarted, or sometimes users log out and then later log back in. This means programs that are executing will stop executing unless there is a way to get the program executing again. As the attacker is not sitting physically at the system, where they can do what they want, it's necessary for the attacker to have a means of getting remote access. This is often done through software the attacker has installed on the system. This software needs to execute, likely when the system boots. This is another case where system services are useful, though scheduled tasks are also useful as are

system and user configurations through techniques like registry keys on Windows systems. Linux systems may use startup files such as `.bashrc` to get programs executed when a user logs in.

- **Privilege Escalation** With privilege escalation, attackers are trying to get the highest level of privileges they can. Most users don't have much in the way of access to their systems. They can get to their data and run programs, but they can't do useful things like install system services or extract system memory. This is where getting additional, elevated privileges is helpful. They may do this by injecting code into an existing process that already has administrative privileges. They may get a service to run at boot time, since the boot process automatically has elevated privileges. There are a lot of TTPs for privilege escalation since it's often a necessary step for an attacker to remain in the environment and gather data, like credentials, they need for further movement.
- **Defense Evasion** Attackers will need to get through a lot of different types of defense. Businesses certainly, though people as well, make use of software on systems to protect against these very attackers. This may be anti-malware or some other detection or prevention software. Attackers need to execute software in ways that won't be seen as malicious or suspicious. This may be using an old-school technique like using a rootkit, where the attacker may be provided remote access while also hiding all of the software being used from the user or other software looking for it. By far, this is the stage or category with the most TTPs. As an example, privilege escalation, an important phase in the attack process, has 12 techniques as of this writing, whereas defense evasion has 37.
- **Credential Access** Applications and systems typically require usernames and passwords to grant access. Attackers will want to try to get these credentials for use elsewhere. This may be through using a phishing attack by sending an email to a victim to get that person to provide username and password to the attacker. It may be from dumping credentials out of memory—from the operating system but also from applications that may be holding on to credentials. Attackers may also be able to use tactics to get the network to give up credentials by forcing authentication against network servers that can provide information to the attacker.
- **Discovery** Attackers are probably not going to be satisfied getting access to just your system. They will be looking for additional systems to get access to. They will want to look around at what you have access to, from systems to applications. They will be gathering a lot of detail to see what else they may be able to access within the target environment.
- **Lateral Movement** One of the reasons for doing discovery, in addition to seeing what data you may have access to that may be stolen for profit, is to determine what other systems may be attacked from your system. The attacker may make use of you and whatever your system has access to. The idea here is to compromise a lot of systems because more systems means more data that could be used or stolen. This may be through the use of existing credentials that have already been stolen. It may be remote exploitation of another system. It may be

through the use of the email of the current victim to contact other potential victims.

- **Collection** Just as with discovery, collection is about gathering information. This may be information that could be used to attack other systems, or it could be information the attacker wants to steal (exfiltrate) for sale or use somewhere else. Once an attacker starts to collect data, this may be more visible than some of the other stages because they are leaving behind artifacts. After all, while they are collecting data, that data has to be stored somewhere. Because the attacker will be storing information temporarily ahead of exfiltration, files with strange names or other distinguishing characteristics may start showing up in places.
- **Command and Control** Keep in mind that attackers will want to maintain access to the systems for as long as possible. After all, if they have targeted your organization, you have something they want. It's likely you will continue to create this thing they want, whether it's intellectual property or customer information or usernames and passwords or credit card data. There's no point in wasting an opportunity by getting in and then leaving. This means they need to retain remote control over the entire environment. This stage is where they set up remote access to the systems in the environment. They may make use of commonly used protocols like the Hypertext Transfer Protocol (HTTP) and abuse it by embedding commands between the server they control and systems in your environment. They can also misuse other protocols like the Domain Name System (DNS) or the Internet Control Message Protocol (ICMP).
- **Exfiltration** Even if they continue to remain in the environment, they will want to get data they have collected out. This is where they exfiltrate the data. They, again, may make use of commonly used protocols since they are less likely to be suspected. The goal is always going to be evading detection as best as possible. However, even when they use commonly used protocols, you can detect the existence of the attacker by following the TTPs provided in the framework. If an attacker is sending data out using DNS, for instance, the message size is likely to be unusually large, so looking for those unusually large messages may let you catch the attacker in action.
- **Impact** This is where the attacker may decide to burn the house down, as it were. Some attackers will get what they want and then tear everything down as they leave. Others may do the same if they happen to be caught. As you are trying to get them out of your network and systems, they are tearing down as much as they can on the way out. This may be destruction of data, disk wiping, encryption of data (meaning ransomware attacks would fall into this category), denial-of-service attacks or other, similarly destructive actions.

The TTPs referenced in the matrix, which you can find on the MITRE website (<https://attack.mitre.org/matrices/enterprise/>), are regularly updated as more become known. Each TTP will have details associated with it, and MITRE will provide mitigation steps to prevent the attack from succeeding to begin with. You may also get advice on how to determine whether the attacker is making use of that TTP.

Security Technology

Invariably, security programs require some technology to implement. The number of technology solutions that may be used within an enterprise continues to grow. The best way to protect an enterprise is no longer about putting a firewall out in front of the network and considering yourself protected, even if that were ever the reality it was thought to be. Today, the attack vectors have changed a lot from what they were even a decade ago. Today's technical solutions are multilayered and aren't entirely focused on prevention.

The assumption today is that prevention isn't possible if the expectation is that 100 percent of attacks will be repelled. As a result, detection is essential. Even detection solutions are multilayered because of the number of entry points into the network. Since detection is a passive effort, it needs to be followed by an active one. This can be incident response, and incident response may require automated collection of artifacts. All of these may require different technology solutions.

Firewalls

The firewall is a traditional security device in a network. Just saying that a firewall is in place, though, doesn't really explain what is happening because there are several types of firewalls running up the network stack. A firewall, in its original meaning, was a wall that kept fires contained. You implemented a firewall to keep fires contained to sections of a building. The same is true for the firewall in a car. The engine compartment is combustible, considering the fuel, oxygen, and electricity mixture there. Car manufacturers put a firewall in to contain any fire that may break out so it doesn't spread to the passenger compartment. The term was taken in the late 1980s to apply to nascent technology being used to protect networks.

Packet Filters

At a basic level, a firewall is a packet filter. Lots of devices offer the capability to filter packets, which is sometimes accomplished with access control lists. Routers and switches will often have the ability to perform packet filtering. This packet filtering is sometimes implemented in an access control list. Packet filters make determinations about the disposition of packets based on protocol, ports, and addresses. Ports and addresses can be filtered based on both source and destination.

Packets can be dropped, meaning they don't get forwarded on to the destination, and there is also no response message sent to the originating system. They can also be rejected, meaning they won't be sent to the destination, but the rejecting device, if it is following protocol specifications, will send an ICMP error message to the sender indicating that the destination is unreachable. This is not only the polite approach, since drops will incur retransmits; it's also generally considered the correct approach from the protocol standpoint. However, when it comes to system security, dropping messages may just make more sense. If messages just get dropped,

it's unclear to the sending system what happened. The target system is essentially in a black hole.

Of course, packets can also be accepted. This can be done as a matter of rules, or it may be policy. Packet filters may have a policy that governs the behavior of the filter. This means there is a blanket rule that applies to everything unless exceptions are applied. A default deny policy, the most secure way to implement a packet filter, will drop everything that isn't explicitly allowed through. You may also have a default accept policy, which means everything is allowed through unless something is explicitly blocked.

You may have run across packet filters if you run Linux systems. While the host-based firewall included in most Linux distributions has other capabilities, it can also function as a basic packet filter. You can set a policy on different chains with the iptables firewall that is in the Linux kernel. As an example, the following lines show running iptables to set a default deny policy on the INPUT, OUTPUT, and FORWARD chains, which are collections of rules applied to specific message flows.

```
[iptables Policy Settings]
iptables -P INPUT DROP
iptables -P OUTPUT DROP
iptables -P FORWARD DROP
```

Packet filtering is basic in its functionality. While these packet filters can be good for inbound traffic or even for keeping internal users from accessing specific ports or IP addresses, they are not good for more complex filtering, such as allowing inbound traffic that is a response to messages that originated on the inside of the network. For this, we need to keep track of those outbound connections, so we need something more than just a packet filter.

Stateful Filtering

Not long after the initial development of packet filters came the development of stateful firewalls. The first stateful firewall was developed in the late 1980s, just like packet filters were. These are firewall types we should know well because they have been around for about three decades at this point. This does not mean, though, that these stateful filters have been in use all that time.

A stateful firewall keeps track of the state of messages within the conversation between client and server. This means the firewall has to have a state table, so it knows about all of the traffic flows passing through it. In the case of the Transmission Control Protocol (TCP), it's theoretically easier since the flags tell the story when it comes to the state of the traffic flow. A message that has just the SYN flag turned on is a NEW connection. It remains in this state until the three-way handshake has been completed. At that point, the state of the flow becomes ESTABLISHED. In some cases, you may have message flows that are RELATED. As an example, the File Transfer Protocol (FTP) will sometimes originate connections from the inside to the outside, meaning from the server to the client. In this

case, the server-to-client connection for transferring the file is related to the control connection from the client to the server.

Even with TCP, the flags don't tell the whole story. After all, it would be easy enough to send messages with the correct flags set to get through a firewall that was only looking at the flags to determine what the state of the communication is. The User Datagram Protocol (UDP) has no state that is inherent to the protocol. This makes it impossible to look at any flags or headers to infer state. Stateful firewalls don't just look at the flags or headers, however. They keep track of all the communication streams so they aren't relying on the protocol. They watch messages coming in and going out and note them along with their directionality to determine what the state is. This means the firewall knows which end is the client and which end is the server.

When you have a stateful firewall, you not only can make decisions based on the ports and addresses, you can also add in the state of a connection. For example, you can see a pair of iptables rules in the following code listing that allow all connections that are NEW or ESTABLISHED into port 22, which is the Secure Shell (SSH) port. Additionally, connections that are established are allowed out on interface eth0. With a default deny policy, new connections won't be allowed out of the interface.

```
[iptables State Rules]
iptables -A INPUT -i eth0 -p tcp --dport 22 -m state --state \
NEW,ESTABLISHED -j ACCEPT
iptables -A OUTPUT -o eth0 -p tcp --sport 22 -m state --state ESTABLISHED \
-j ACCEPT
```

This gets us a little further in our capabilities to make decisions about what packets to allow or deny into our networks. While it doesn't seem like a lot, these capabilities provide a lot of potential for keeping bad people out.

Deep Packet Inspection

One of the biggest issues when it comes to firewalls is that they allow known services through. A firewall will allow connections to a web server through. This means attack traffic to the web server will just be let through the firewall. Attack traffic that uses the application against itself will set up connections just like any other client. From the standpoint of the packet filters and stateful filtering, the messages will pass through. Because of that, we need to go deeper. Just looking at the protocol headers is insufficient, because everything will look correct and legal in the headers. We need to start looking at higher layers of the stack.

A deep packet inspection (DPI) firewall looks beyond the headers and into the payload of the packet. With this approach, it's easier to identify malware and other inbound attacks. A DPI firewall would require signatures that it should look for in the packet to determine whether something is going to be malicious so it can block the traffic from coming into the network. To do this, the firewall has to parse the entire message before it can make any determinations. This means it has to have at least the entire

packet, meaning any fragmentation at the IP layer has to arrive and be re-assembled. In some cases, it may need the entire stream, whether that's UDP or TCP. This certainly means a little latency on the arrival of messages.

Packet filters and stateful firewalls don't need to reassemble anything, assuming the entire header has arrived. Any message that doesn't have the entire set of IP and TCP/UDP headers is likely a problem anyway, since that's well under 100 bytes. Fragmenting a message at under 100 bytes could be a result of malicious traffic, trying to fool a firewall or other security solution. They belong together. Only looking at the headers is limiting, though, especially since so many attacks today come in over the higher-layer protocols. A DPI firewall provides the ability to inspect those higher-layer protocols.

One consideration to keep in mind, though, is that encrypted traffic can't be inspected. The firewall won't have the key to decrypt the message to look at it. Any man-in-the-middle approach to encryption on the part of the firewall violates the end-to-end expectation of most encryption solutions and users. The headers, of course, aren't encrypted. If they were, no intermediate device would be able to determine where anything was going. This means packet filters and stateful firewalls are not impacted by encryption. DPI firewalls, though, are. This means that with the move to encryption over all web-based connections, DPI firewalls don't do a lot to protect the web server.

Application Layer Firewalls

There are application layer firewalls in addition to the DPI firewalls. While these firewalls also inspect the packet, they commonly are specific to a particular protocol. For example, in voice over IP (VoIP) networks, a device called a *session border controller* (SBC) can be used. This is a device that understands the VoIP protocols—commonly either H.323 or the Session Initiation Protocol (SIP). As such, it not only can make determinations about the validity of the messaging but also can open up dynamic pinholes to allow the Real-time Transport Protocol (RTP) media messages through, since they would be over different ports and protocols than the signaling messages would be.

An SBC would be an example of an application layer firewall, since it has the capability of making decisions about allowing traffic through. It makes these decisions based on understanding the application layer protocol and common state flows of that protocol.

Another common application layer firewall would be a web application firewall (WAF). The WAF uses a set of rules to detect and block requests and responses. Given the number of web-based attacks, keeping up with these rules can be challenging. While there are several commercial WAFs, there is also ModSecurity, which is an open source module that can be used with Apache web servers. The rules can get complicated, and you can see an example in the following code listing.

```
[mod-security Rule]
    SecRule RESPONSE_BODY "@rx (?i)(?:supplied argument is not a valid
MySQL|Column count doesn't match value count at row|mysql_fetch_array\(\)|on
MySQL result index|You have an error in your SQL syntax;|You have an error in
your SQL syntax near|MySQL server version for the right syntax to
use|\[MySQL\]\[ODBC|Column count doesn't match|Table '[(^)]+' doesn't
exist|SQL syntax.*MySQL|Warning.*mysql_.*|valid MySQL result|MySQLClient\.)"
\
    "capture,\
    setvar:'tx.msg=%{rule.msg}',\
    setvar:tx.outbound_anomaly_score=+{%tx.critical_anomaly_score},\
    setvar:tx.sql_injection_score=+{%tx.critical_anomaly_score},\
    setvar:tx.anomaly_score=+{%tx.critical_anomaly_score},\
    setvar:tx.%{rule.id}-OWASP_CRS/LEAKAGE/ERRORS-
    %{matched_var_name}=%{tx.0}"
```

The rule you see looks in the response body for the message that is found after `@rx`. This indicates that what comes next is a regular expression. The regular expression describes the message the rule is looking to match on. If the rule matches, the regular expression match will be placed into the transaction variable collection because of the `capture` action. There will also be several variables that get placed into the transaction variable collection.

In addition to capturing information, which can be logged and referred to later or be used for alerting, ModSecurity can block as an action. This will prevent the message from going past the WAF. What this means is that the WAF sits in front of the web server that is being protected. Sometimes, a WAF like ModSecurity is implemented on the edge of the network as a reverse proxy, so clients send messages to the reverse proxy, which handles the message on behalf of the server, parsing it for potential bad requests, before sending it on to the actual web server. Responses also pass through the reverse proxy.

These are just a couple of examples of application layer firewalls. They may also be called application layer *gateways*. Any device that can make decisions based on what is happening in the application layer protocol and then have the ability to drop the message, regardless of the application layer protocol, could be considered an application layer firewall.

Unified Threat Management

Sometimes a firewall alone isn't enough. Even in the case of application layer firewalls, you still need to protect the users. Users are often the most vulnerable point on your network, and they are regularly targets of social engineering and malware attacks. A unified threat management (UTM) device is one that consolidates a lot of security functions into a single system that may be placed at a single point in the network. This UTM would replace the firewall, intrusion detection, and intrusion protection devices as well as offering antivirus protection.

There are downsides to this type of approach. You now have a single point in your network where all of your security is handled. If this device fails for whatever reason, you have no backstops in place. If your firewall

failed, for instance, you would still have intrusion detection systems to catch anything that got through, if that had happened. With UTM, you have one place, and it needs to work all the time and be configured correctly all the time.

Intrusion Detection Systems

There are two different types of intrusion detection systems (IDSs) that you can find. The first is a host-based IDS. A host-based IDS watches activity on a local system, such as changes to critical system files. It may also watch log files or audit other system behaviors. One of the challenges with a host-based IDS is that once an attacker is in a position to trigger a host-based IDS, they are on the system, which means they can become aware that the IDS is in place and make plans accordingly.

The second type is a network IDS. Where firewalls have the ability to block or allow packets in the network stream, a network IDS can take some of the same sorts of rules and generate log messages. A rule for an intrusion detection system can generally be based on any layer in the network stack. As long as you can create some sort of identification for the messages you want to log or alert on, you can write a rule for an IDS. As with the firewalls, there are a number of commercial options. There are also some open source options, one of the biggest names being Snort, which is currently owned by Cisco, but free access to the program and some community rules is still offered.

A network IDS watches all network traffic that passes by the network interface. This means that placement is important. There may be different approaches to placement, depending on the IDS product being used. One of these approaches is to place the IDS connected in parallel rather than in series at the very perimeter of the network so all network traffic coming in can be observed. [Figure 3.3](#) shows a simplified diagram using this approach, with the IDS behind the firewall but not in the traffic flow directly. Traffic gets shunted to the IDS so it can detect without getting in the way or causing a network failure. Another approach is to place sensors in different parts of the network so you don't have a single device. This approach may be especially useful if you are also trying to look at potential insider attacks.

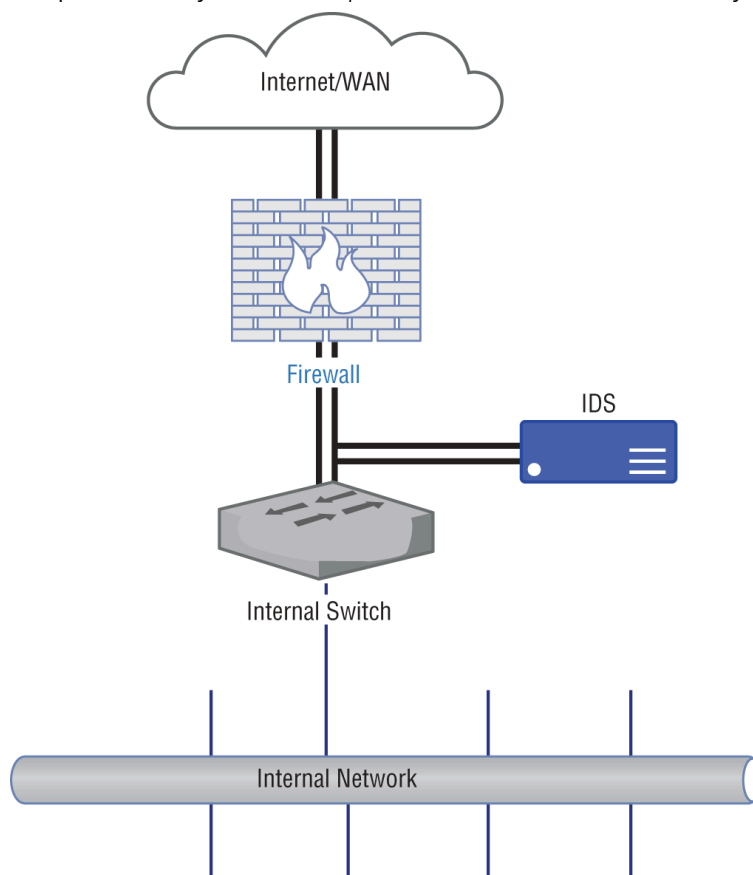


FIGURE 3.3 Network diagram showing IDS placement

It used to be the case, many years ago, that IDS devices had problems keeping up with all of the traffic coming through a network, since the IDS device had to look at each packet and compare it against as many rules as were configured. This isn't the case any longer because processing power has increased considerably, as has overall bus throughput in systems. Even so, using multiple sensors across different parts of the network can allow for sharing the processing load.

We can take a look at Snort rules to see what an IDS can do and how it may work. Just as with WAF rules, it takes some time and effort to understand how to write rules, especially in cases where what you are trying to detect on is content in the packets. In the following code listing, you will see two Snort rules that demonstrate some of what Snort can do.

[Snort Rules]

```

alert tcp $EXTERNAL_NET any -> $SQL_SERVERS 7210 (msg:"SQL SAP MaxDB shell command injecti
alert tcp $EXTERNAL_NET any -> $HOME_NET 21064 (msg:"SQL Ingres Database uuid_from_char bi
  
```

Snort rules start with an action. You can alert, as is done in both of these rules, which generates an alert message to whatever is configured for output. In addition, however, you can log, drop, reject, and pass, along with some other actions. Some of these capabilities take us beyond traditional IDS functionality, but they are actions that Snort can take in rule configurations. After the action, you configure the details about the headers. This includes the protocol you want to alert on. In our case, we are alerting on a TCP message, so we need to specify not only a source and

destination address but also a source and destination port. The `->` (arrow) indicates the direction of the flow.

Inside the parentheses are the details about the rule. First is the message to use in the log. After that, you will find details about the flow. Snort is aware of which side is the client and which is the server, so you can indicate which side the message should come from. While you've specified source and destination ports, this is a little more detailed. The important part comes next, where we specify what the packet should contain to match on this rule. You will also see metadata, such as reference information. This reference information may include details about a vulnerability this is meant to alert on. Finally, rules will have a Snort identification number (SID). While this is configurable, there are conventions about numbers that should be used for user-defined rules. Values up to 999999 are reserved for the use of rules that come with the Snort distribution. You can also specify a revision number, so you can have some version control on the SIDs you use.

Commonly, you would use alert or log in the case of an IDS, since the intention of an IDS is just to detect events. The alert assumes that someone is paying attention to what the IDS is emitting and taking actions based on the alert and a subsequent investigation. There are, though, cases where you may want the IDS itself to do something with what it found.

Intrusion Prevention Systems

An intrusion prevention system (IPS) takes an IDS a step further. As noted earlier, Snort has actions including drop and reject. These are actions that are beyond the capability of Snort itself. Snort monitors network traffic and parses through it to identify packets that contain possibly malicious contents, as identified by the rules. To have Snort run as an IPS, it has to be placed inline, meaning it has to be a device in the path of traffic into and out of the network. Any IPS would have to be similarly configured in order to have the ability to block any network traffic. [Figure 3.4](#) is a simplified network diagram showing the potential placement of an IPS.

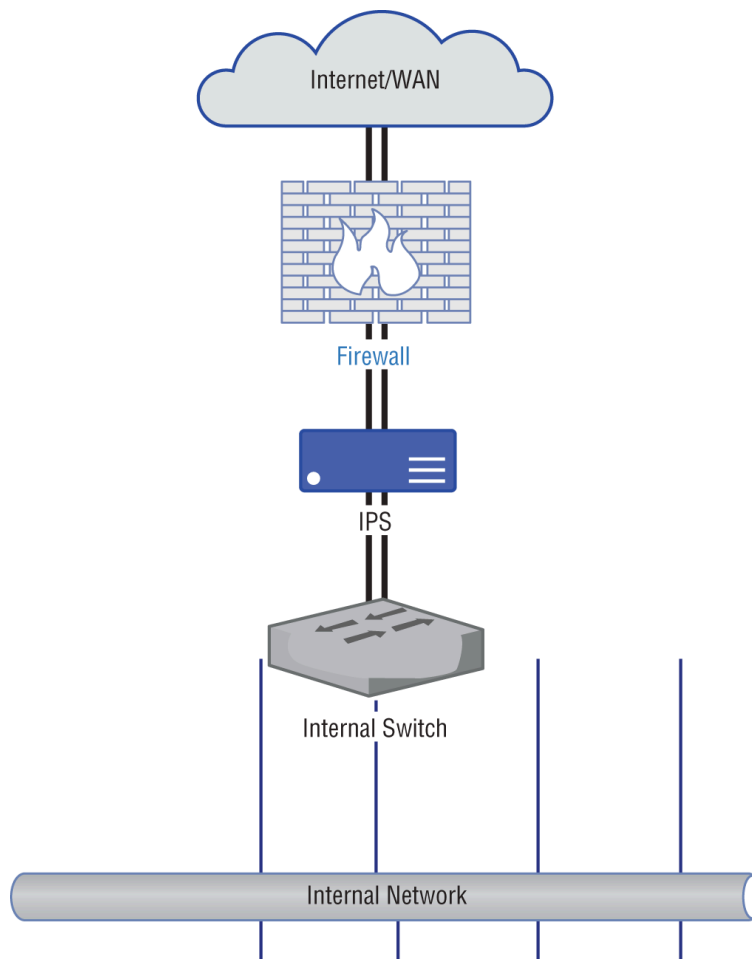


FIGURE 3.4 Network diagram showing IPS placement

With the IPS in the flow, it can act like a firewall, making decisions about whether to accept or reject packets as they enter the network. The difference between an IPS and a firewall is that the “firewall rules” on an IPS would be dynamic. Rather than having large blanket rules blocking IP addresses wholesale, the IPS would make decisions based on the contents of the packet. The rule would be in place for the duration of the packet passing through the IPS—essentially a one-off. The rule may just exist to handle that one packet, or it may be in use for a longer period, but the rules are dynamic and temporary by nature. Also while inline, either the IPS can choose to drop the message, meaning just discard it, or it can reject it with an appropriate message. This may be an ICMP destination unreachable message, for example.

Even with an IPS where potential attacks are blocked, the logs and alerts should be investigated. Not every message that matches an IDS/IPS rule will be a legitimate attack. In several cases of running a basic Snort installation with the community rules, I have run across alerts indicating bad SIP messages, when in fact the traffic was HTTP over port 80, not SIP. Rules are not infallible, so it is possible for rules to catch legitimate traffic, and if you are running an IPS, you may end up blocking messages from customers or partners.

One of the challenges of an IDS or IPS is the volume of logs they can create. This is a factor of the number of rules you have, how well they are written, and how much traffic you get into your network, as well as the

placement of your sensors. Alerting is a significant challenge, and it can require a lot of work to fine-tune rules so your analysts, or whoever is looking at the alerts, don't become screen-blind, meaning they get so used to seeing the same alerts over and over that they end up missing one that is legitimate—a white rabbit in a field of white snow/noise. Fortunately, there are solutions that can help with that problem.

Endpoint Detection and Response

While you may be familiar with anti-malware software, used to detect the existence of viruses and other malicious software on a system, you may be less familiar with this. Endpoint detection and response (EDR) is a class of software that can perform a range of functions that are useful to a security operations staff. There are several commercial offerings in the EDR space, and each one of them may provide a different set of functionalities, though there will be a core set of functions they all provide.

One function they may provide is anti-malware. Not all EDR solutions will offer this sort of preventive control. An EDR that offers anti-malware may be focused not only on the signatures that would be typical for anti-malware but also behavior-based detections. After all, the vendors of this EDR solution are aware of the same TTPs that MITRE publishes so they can look for the TTPs that would relate to how malware may expect to install and run. They may be able to prevent malware from executing, or at least generate an alert to indicate suspicious behavior was found.

EDR solutions may also detect other malicious behavior, which may be done through investigating logs that are available on the system. However, the most common function EDR tools will perform is being able to remotely assess systems from a central console. Additionally, these tools can be used to pull back artifacts from the systems. This may include such details as process listings, memory dumps, files, or logs. [Figure 3.5](#) shows the web interface from Google Rapid Response (GRR), which is an open source EDR tool. It is not as feature rich as some of the commercial tools available, but it is free and can be used to easily extract data from remote systems.

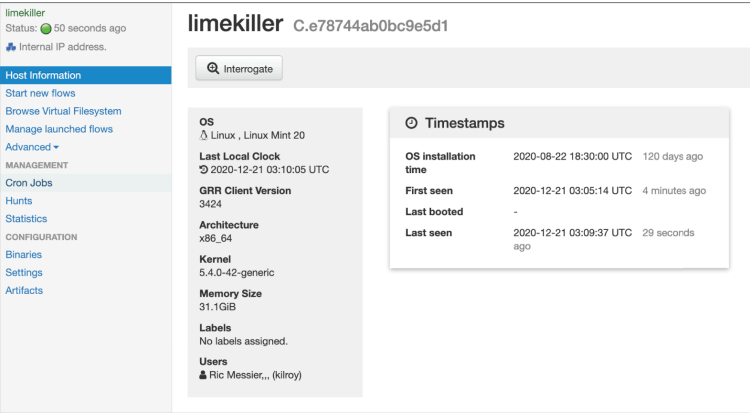


FIGURE 3.5 Google Rapid Response system

Having the ability to capture details quickly without having to go to the target system makes response a lot faster. However, attackers do know

about these tools and may simply opt not to run if the existence of the tool is detected. They may also go into more extreme versions of evasion to make it a lot harder for the EDR to locate the attacker and their techniques. However, if you need to be able to capture a lot of details about a remote system for any sort of investigation, these tools are invaluable.

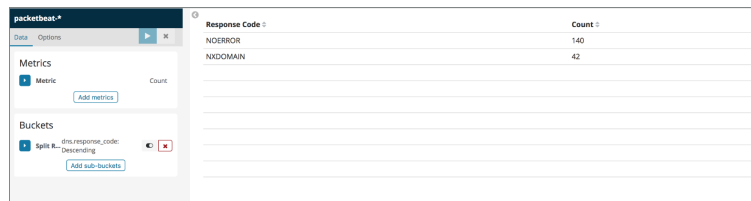
Another function EDR tools can often provide is isolation. If you happen to notice an attacker has gained access to a system, you want to keep them from being able to get to other systems within the environment. You don't want them to know that you know they are there, however. If they know you know they are there, they either may go into hiding, remaining in the environment while staying away from your prying eyes until you stop prying, or may burn everything down just to make life harder for the business.

An EDR tool may be able to perform something called *host isolation*. While this may be able to be done within the network, especially if you have firewall controls within the network segments, it can be more efficient to just use a software control on the local system where the attacker has been seen. The attacker retains access to the system, but they are unable to get out to any other system. This may be done while the complete scope of the investigation is being completed—trying to figure out everywhere the attacker is within the environment.

Security Information and Event Management

A good practice from the standpoint of both system administration and security is system logging. Logs are helpful to diagnose problems in the system and network. They can also help in an investigation of a potential issue. Forensic investigators and incident responders will find them invaluable. If you've ever had to troubleshoot a system or application that has failed without having any logs to look at, you will understand the value of logs. However, turning up all logging to the maximum isn't the answer either. Logs consume disk space, though disk space is generally inexpensive these days. They also consume processing power, and more logging can actually make it harder to find problems because you're having to wade through enormous volumes of text to find one entry that will help you. You won't always know the text to search for after all.

This is where a good log management system can be helpful. In the case of security incidents, these log management systems can also include search and correlation tools. While there are many log management solutions, many organizations are moving to something called *security information and event management* (SIEM). SIEM software, however, is not simply a log management solution. You don't just dump all your logs into it and move on. SIEM software is used to correlate and analyze security alerts. It will also provide you with the ability to better visualize your data. As an example, in [Figure 3.6](#), you can see DNS response codes in Kibana, which is a part of the Elastic Stack (Elasticsearch, Logstash, and Kibana), formerly known as the ELK Stack.



The screenshot shows the Kibana interface for a packetbeat search. On the left, there are tabs for 'Data' and 'Options', and a 'Metrics' section with an 'Add metrics' button. Below that is a 'Buckets' section with a 'Split By' dropdown set to 'response_code' and a 'Descending' sort order, with an 'Add sub-buckets' button. The main area displays a table of search results.

Response Code	Count
NXDOMAIN	140
NXDOMAIN	42

FIGURE 3.6 Kibana interface to the Elastic Stack

The Elastic Stack is a good platform to ingest large amounts of data from multiple sources. This can include log data as well as packet data. What you see in [Figure 3.6](#) is a listing of the DNS responses that have been seen by the systems reporting to this Elastic Stack installation. Like other SIEM products, Elastic Stack provides a lot of search capabilities. It will also provide a large number of visualization options based on the data that is being provided to it.

A security operations center (SOC) will often be built around monitoring systems, including a SIEM system that can be used to correlate data from a number of different sources. The advantage to using SIEM is being able to pull a lot of data together so you can get a broader picture of what is happening across the network. This will help you to see trends and larger attacks. Seeing an attack in isolation may get you to focus on the one attack you are looking at, rather than recognizing that there are several other systems that are also being attacked. A single system doesn't necessarily provide you with the entry point.

Having data points can help to ensure that the right controls are in place to protect information assets. Data is only a starting point, however. You still need to be able to interpret it and understand what to do with what you have. A strategy is important. This can be policies and procedures as well as a technology plan. You may also want to have a philosophy about how you are going to implement everything you have.

Being Prepared

Technology is all well and good. However, technology in and of itself is insufficient to protect a network. It's been said that the only secure computer (and, by extension, network) is one that has had all of the cables cut and, ideally, has been filled with cement and dropped to the bottom of the ocean. If you want to do anything with a computer, you are exposing yourself to the potential for that computer to become infected or compromised. Security comes from planning and preparation. To be prepared, you need to make sure you have thought through the implications of your actions, as well as how you are implementing any solutions you have in place.

Implementing security solutions requires understanding where your resources are—this is not only information assets. It is also your technology assets. Perhaps most important, it is your human assets. Technology alone won't be sufficient. You can take care of the technology aspects with a defense-in-depth approach to security design and architecture. A defense-in-breadth approach, though, requires humans.

As mentioned earlier, having logs is really important when it comes to responding to security events. While it's nice to have a SIEM solution, even if you have one, you need data to feed to it. You can get that data by making sure you are logging on to your systems. Additionally, accounting information is useful so you have a trail of activity to track behaviors.

Defense in Depth

Defense in depth is a layered approach to network design. Anytime I hear the term *defense in depth*, I think of Minas Tirith in the *Lord of the Rings* books. Minas Tirith is a city with seven concentric walls. Inside the seventh wall is the Citadel. If an invader somehow manages to breach one of the walls, the people fall back into the next ring until, if necessary, they end up inside the final wall. Ideally, invaders will either give up or be defeated as they try to make it through seven enormous walls. As discovered in the book *The Return of the King*, though, even seven walls aren't always sufficient for defense of the city.

The reason Minas Tirith comes to mind, in case it's not clear, is because it's an example of defense in depth. Using a defense-in-depth approach, you would design a layered network with multiple security gateways to control access between layers. One of the objectives of a defense-in-depth approach is to delay the attacker or adversary. The goal isn't to prevent the adversary from getting in necessarily. It's to cause delays. It's also to, ideally, create artifacts along the way. When a defense-in-depth strategy is used, as an adversary moves through the layers of the network, they leave traces. These traces can be detected by automated analysis tools or operations staff. The hope is that the detection happens, leading to shutting down the attack, before the adversary gets to any really sensitive information.

Defense in depth is a military concept that has been adapted for other uses. One of them, of course, is information security. One thing you will notice that you get from an approach like a multiwalled city is redundancy. The second wall is redundant in relation to the first. The third is redundant to the second (and by extension the first), and so on. This doesn't necessarily mean that you just keep placing the same controls back to back everywhere. A better approach is to provide redundant controls across multiple areas.

The first area of controls to consider is physical. This means ensuring that unauthorized people can't get physical access to a system. Anyone with physical access to a system has the potential to get logged into the system—as an example, there may already be a user logged in, and if the system isn't locked, anyone could become the logged-in user. The attacker may be able to boot to an external drive and change the password or create another user. There are a few ways for an attacker to get access to a system if they have physical control of it.

The second area of controls is technical. This is what I've been talking about—firewalls, intrusion detection systems, SIEMs, and other security technology. It's not just that, though. It can be software or configuration controls like password policies, requiring complex passwords. Antivirus

or other endpoint protection would also be a technical control. Any hardware or software that is in place to prevent unauthorized use would be a technical control.

Finally, there are administrative controls. These are the policies, standards, and procedures. This may include policies or practices like hiring standards, background checks, and data handling policies. These administrative controls set the tone for how the organization is protected, and they are essential.

Figure 3.7 shows an example of how some elements of a defense-in-depth approach might be implemented. You can see at the entry point to the network that there is a firewall. The firewall then has two separate networks that come off it. The first is a demilitarized zone, which is where servers that are untrusted but still part of the corporate network reside. This is where you might place Internet-facing servers like web or email. The other side of the firewall is where you might put internal servers like Active Directory or file servers. This sort of network segmentation is often part of a defense-in-depth strategy.

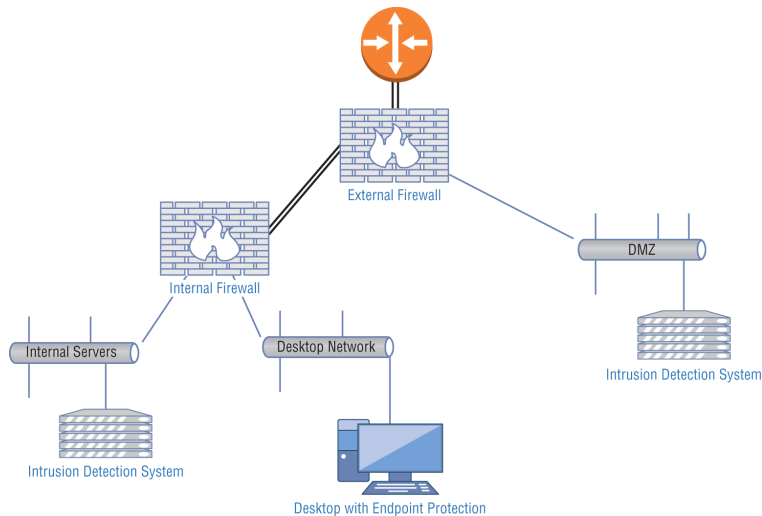


FIGURE 3.7 Defense-in-depth network design

A second firewall protects the internal network from the server network and vice versa. Finally, we have intrusion detection and endpoint protection in place. What isn't shown in this diagram are the procedures and policies that are in place in the business. We not only have several tiers in the network that an attacker would have to traverse, we also have different detection and protection points. Of course, there is more to it than even just the network design, the technology, and the policies and procedures.

Defense in Breadth

For several years now, there has been something of a debate between the advantages of defense in depth versus those of defense in breadth. Is defense in depth sufficient, or do organizations need to be implementing defense in breadth? Of course, the challenge to defense in breadth is that it is rarely explained very well. If you search for explanations of defense in

breadth, you will find a lot of rationale for why defense in breadth is a good thing and why defense in depth in and of itself is insufficient for a modern strategy to implement security solutions within an organization. The problem is that when you can find a definition of what defense in breadth is, it's very vague.

My understanding of defense in breadth over the years has been that it's all the surround that is missing from defense in depth. Defense in depth doesn't take into account all of the human factors. Defense in breadth is meant to take a more holistic look at the organization from a risk perspective. The organization is meant to evaluate risk and take a systemic look at how to mitigate any risk, taking into account the threats that the organization expects to be exposed to. To fully evaluate risk and take a comprehensive look at potential solutions, an organization needs to have data.

One way, perhaps, to think about it is that defense in depth is really about prevention. However, prevention isn't entirely realistic. This is especially true in an era where the old technical attacks of exploiting vulnerabilities in listening services exposed to the outside world are no longer the vectors of choice. Instead, going after the user is more productive, so social engineering attacks and a lot of malware are more common. As a result, the best approach to security is not to focus primarily on prevention but instead assume that you will get breached at some point and prepare for a response.

To be prepared for a response, you need a lot of detection and historical data. Additionally, you need an incident response team whose members know what their roles are. You also need a lot of communication across different teams. The members of the security team are not always the best people to perform all the work in response to events. This means that breaking down some of the traditional silos is essential.

A growing trend in the information technology (IT) space is the collaboration between development teams and operations teams, commonly referred to as DevOps. Companies that are even more forward-thinking are also ensuring that security is merged in along the way. What you end up with is something that is often called DevSecOps. These approaches focus not only on teamwork and the associated communication that is necessary but also on automation. Automating builds, testing, and deployment takes a lot of the human factor out, helping to avoid misconfigurations or other mistakes.

You can see that defense in breadth can be a very complicated idea that may be difficult to fully implement once you start thinking about where you can inject security conversations into your organization. Ultimately, though, the goal of defense in breadth is to be better positioned to not only protect the organization but also respond to attacks—also, to respond to a breach with the goal of restoring business operations as quickly as possible.

Defensible Network Architecture

Another approach to consider when putting a network together is to use a defensible network architecture. This does not preclude either defense in depth or defense in breadth. Instead, it focuses on building your network in a way that you can more easily monitor and control. Keep in mind that one of the techniques an attacker will use is to move laterally (from system to system) within a network environment. This means once they have compromised one system, they will try to move to another system. Since it's easier to see systems on the local network, they may target those systems for a while until they get some additional information about other systems on other networks.

Attackers don't generally have the big picture with regard to the entire enterprise network, unless they happen across a network diagram. If you create a network that has firewalls at the entry/exit points of the network, a common approach used to segment the network, you have the ability to monitor all traffic going into and out of the network. You also have the ability to create a choke point, meaning if you suspect an attacker is in one network segment, you can restrict all access to other network segments.

Having these types of controls of blocking and monitoring allows you to better keep track of and control over what an attacker is doing, if you catch them late in the attack lifecycle and they have taken up residence to at least a small degree. You may not have blocked them from getting initial access, but you should have the ability to keep an eye on them and restrict further access while you get the complete scope of their infiltration of your environment.

Logging

One idea that has arisen in several places along the way here is logging. This is not only system logging but also application logging. Beyond that, it's not just logging on systems—desktop or server—but also on network equipment like routers and switches and certainly firewalls. Even in cases where you are running an IDS in the network, you may not always want to alert, because there may be some events that you don't feel it necessary to follow up on. However, the fact that the event happened may be useful to know about once a breach has happened. The importance of having historical data can't be overstated.

Many systems, including Unix-like systems as well as network devices, will support the syslog protocol. This is a logging protocol that began as the logging mechanism for the Simple Mail Transfer Protocol (SMTP) server sendmail. Over the years since the 1980s, when syslog was first implemented in sendmail, it has become the standard logging solution on Unix-like systems, and the protocol has been documented in RFC 3164, later standardized in RFC 5424. Syslog not only has an easy-to-understand syntax in the creation and reading of messages, it also can be used for remote logging as well as local logging.

Because syslog can support remote logging, it can be used as a centralized log host. This fact is, perhaps, especially important when preparing for incident response. A common approach to wiping your tracks is to wipe logs. Any attacker who gets access to a system may be able to wipe the local logs, but if they are streaming off the system, they are retained for later use, unless the attacker can compromise the central log host.

You can see an example of syslog messages in the following code listing. Each message starts with the date, followed by the originating hostname. After that is the process that created the log, including the process identification number. Finally, the message that the process generated is shown.

```
[syslog Messages]
Jun 26 10:27:16 boardinghouse kernel: [923361.001444] vmbr0: port 3(tap210i0) entered forv
Jun 26 10:27:17 boardinghouse pvedaemon[10864]: <root@pam> end task
UPID:boardinghouse:000034F1:0580EE23:5B326963:qmstart:210:root@pam: OK
Jun 26 10:27:42 boardinghouse pvedaemon[9338]: <root@pam> starting task
UPID:boardinghouse:00003552:0580F8B5:5B32697E:vncproxy:210:root@pam:
Jun 26 10:32:09 boardinghouse pvedaemon[9338]: <root@pam> end task
UPID:boardinghouse:00003552:0580F8B5:5B32697E:vncproxy:210:root@pam: OK
```

The syslog standard defines facilities to categorize messages so they can be routed to different locations. Any messages related to authentication, for example, can be put into a single file, away from other types of messages. Each facility can be routed to a different file or even a different system, if you wanted some log messages stored locally and others stored centrally. In some syslog implementations, you can store both locally and remotely. This provides local logs as well as remote logs that function as backups.

Beyond facilities, syslog defines severity. This means applications can determine the level of an event, including informational, debug, warning, and error. Just as with facilities, severities can be redirected to different locations.

Not every system is Unix-like, of course. Windows systems are very common, on the desktop as well as the server. On Windows systems, log messages get sent to the event subsystem by default. Instead of the text-based messages that syslog uses, the event subsystem uses a binary storage system. Of course, one advantage of the way the event subsystem stores data is that it can be queried, as though it were a database. While it's not as easy to parse messages, the fact that you can query messages means you can easily return related messages with a single query.

The event subsystem in Windows also has a large amount of data. In addition to the different categories, such as system, security, and application, there are event IDs. An event ID can identify all entries that are the same. Searching for the event ID can give you every instance of a particular event, which may occur across multiple processes. [Figure 3.8](#) shows a single event from the Windows Event Viewer where you can see the event ID. You can also see other pieces of information that come with each event in the Event Viewer.

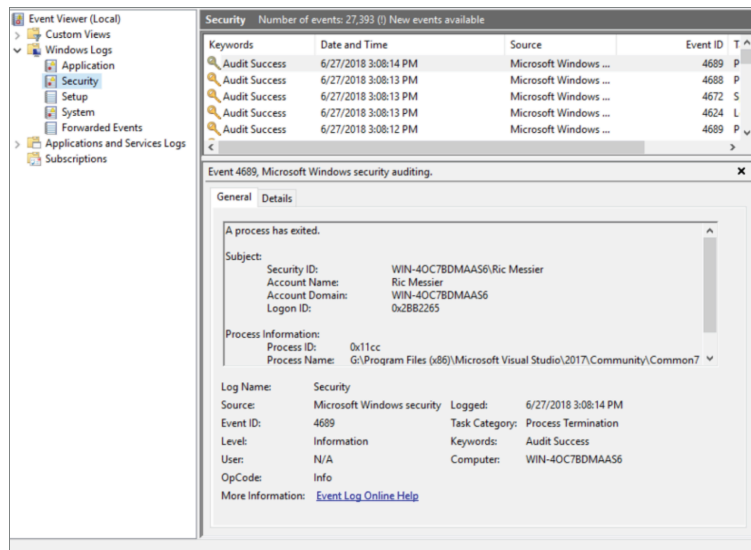


FIGURE 3.8 Event Viewer

In both cases, application developers can make use of the system-provided logging functionality. On the Unix side, any developer can write to syslog and the syslog service will take care of writing the logs out based on the definition in the configuration file. This takes the onus of developing logging schemes off the developers and also ensures a single place to go for log messages. The same is true on Windows systems. Any application developer can make use of the event subsystem, using it to write log messages. The application can even have its own log file and Windows will break the logs out to separate locations. If you were to go into the Event Viewer application, you would be able to look at all the event log entries and the different categories the event logs fell into. You would typically see a list of all the applications Windows is logging for. Any application could have its own entry in that list to keep the logs separate from the system while it's still logged to the same location as all the other logs.

Auditing

Logs are important, and we generally rely on application developers to implement logging and, ideally, configurable logging, meaning different levels of log messages. Being able to turn up logging to a verbose level can make troubleshooting problems much easier. The same is true for the operating system. This is where auditing can come in. Both Unix-like systems and Windows systems offer the ability to enable and configure auditing. Of course, the definition of auditing is different across the two systems.

On Windows systems, auditing is a security function. It relates to success or failure of system events. This can include success or failure of logins, for example, or access to files on the system. [Figure 3.9](#) shows the settings of the audit policy within the Local Security Policy application. Each category in the policy can have success or failure logged. This isn't an either/or, though. You can log both success and failure. If the audit events are triggered, they will show up in the security log in the Windows Event Viewer.

On the Linux side, there is a completely different auditing infrastructure. Using the program `auditctl`, audit policies can be managed on a Linux system. The auditing subsystem in the Linux kernel can be used to watch files and directories for activity. It can be used to monitor application execution. Perhaps most important, it can also be used to monitor system calls. Any system call used by any program on the system can be monitored and logged. The audit subsystem typically has its own set of logs. An example of some of the log entries from `audit.log` on a CentOS Linux system is shown in the following code listing.

[audit.log Sample Output]

```
type=USER_LOGIN msg=audit(1530130008.763:341): pid=9711 uid=0 auid=0 ses=30 msg='op=login
type=USER_START msg=audit(1530130008.765:342): pid=9711 uid=0 auid=0 ses=30 msg='op=login
type=CONFIG_CHANGE msg=audit(1530130271.424:353): auid=4294967295 ses=4294967295 op=add_ru
type=SERVICE_START msg=audit(1530130271.424:354): pid=1 uid=0 auid=4294967295 ses=42949672
type=SYSCALL msg=audit(1530130283.962:355): arch=c000003e syscall=59 success=yes exit=0 a0=
type=EXECVE msg=audit(1530130283.962:355): argc=2 a0="cat" a1="audit.log"
type=CWD msg=audit(1530130283.962:355): cwd="/var/log/audit"
type=PATH msg=audit(1530130283.962:355): item=0 name="/usr/bin/cat" inode=2799 dev=fd:00 m
type=PATH msg=audit(1530130283.962:355): item=1 name="/lib64/ld-linux-x86-64.so.2" inode=3
type=PROCTITLE msg=audit(1530130283.962:355): proctitle=6361740061756469742E6C6F67
```

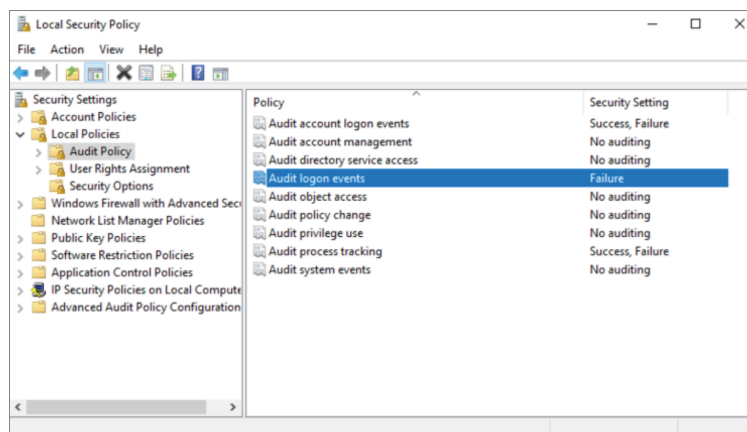


FIGURE 3.9 Audit Policy in Windows

Each entry provides details about the type of the entry, which indicates what the audit subsystem has detected. For example, the first entry indicates that a user has logged in. You can see from the details that the address the connection came from is 192.168.86.49, that the executable is `/usr/sbin/sshd`, and that the result was a success. Other entries indicate file actions, program executions, and configuration changes.

Much as with the auditing under Windows, the auditing under Linux needs to be configured. You do this by creating rules that are used by the audit daemon. You can make the changes directly in the configuration files, or you can add the rules on the command line using the `auditctl` command. This implements changes directly and is what gets called as the audit system is coming up. What go into the configuration file are the command-line parameters that would get passed to `auditctl`.

Summary

There are some essential concepts in information security that we need to get behind us. The first are the three elemental properties of information security—confidentiality, integrity, and availability. Confidentiality is the need to keep secret information secret. Integrity means ensuring that data doesn't change when it isn't expected to change. Finally, availability means information, including services, is available when it is expected to be available. Beyond the triad, as confidentiality, integrity, and availability are known, is the Parkerian hexad, which adds in utility, authenticity, and possession or control.

There are technology elements that are commonly implemented. One you will see almost everywhere is a firewall. Firewalls come in multiple areas of functionality, however. The most basic firewall is packet filtering, meaning decisions can be made based on packet headers. Stateful firewalls factor in the state of the connection when it comes to decisions about allowing or blocking traffic. Deep packet inspection firewalls can look at the data payload to determine whether to block messages. Finally, unified threat management devices, which some call next-generation firewalls, can take firewall functionality to a new level by also adding in antivirus. The antivirus looks at the network stream for malware rather than waiting until it gets to the endpoint.

Application layer gateways are also a variation of a firewall. These are devices that are aware of the application layer protocols and can make decisions about the packets based on whether the traffic matches to allowed endpoints and whether the flow is correct based on the protocol.

Technology alone isn't enough. A defense-in-depth strategy applies layers not only to the placement of the technology but also to the policies, procedures, and standards that are required to guide the security functions. Defense in breadth adds in a broader view to defense in depth with a lot of additional surround across the organization. This may include awareness training, detection, preparation for response, and other capabilities that are beyond just preventing attacks.

Preparing for attacks is essential in businesses today. Certainly there are incident response plans, policies, and teams. However, from a technology standpoint, logging and auditing can be very helpful when it comes time to respond. Logging may be system logging or application logging, where there is a trail of actions that have been determined by the programmer. When it comes to auditing, Windows auditing capabilities include success or failure logs for access to files, users, or other system objects. On the Linux side, audit rules can be written to watch files and directories as well as system calls.

Review Questions

1. To remove malware from the network before it gets to the endpoint, you would use which of the following?
A. Packet filter

- B. Application layer gateway
 - C. Unified threat management appliance
 - D. Stateful firewall
2. If you were on a client engagement and discovered that you left an external hard drive with essential data on it at home, which security principle would you be violating?
- A. Confidentiality
 - B. Integrity
 - C. Nonrepudiation
 - D. Availability
3. How would you calculate risk?
- A. Probability * loss value
 - B. Probability * mitigation factor
 - C. (Loss value + mitigation factor) * (loss value/probability)
 - D. Probability * mitigation factor
4. Which of the following is one factor of a defense-in-depth approach to network design?
- A. Switches
 - B. Using Linux on the desktop
 - C. Optical cable connections
 - D. Access control lists on routers
5. How would you ensure that confidentiality is implemented in an organization?
- A. Watchdog processes
 - B. Encryption
 - C. Cryptographic hashes
 - D. Web servers
6. An intrusion detection system can perform which of the following functions?
- A. Block traffic
 - B. Filter traffic based on headers
 - C. Generate alerts on traffic
 - D. Log system messages
7. Which of these would be an example of a loss of integrity?
- A. User making changes to a file and saving it
 - B. Bad blocks flagged on disk
 - C. Credit cards passed in cleartext
 - D. Memory failures causing disk drivers to run incorrectly
8. What would you use a security information event manager for?
- A. Aggregating and providing search for log data
 - B. Managing security projects
 - C. Escalating security events
 - D. Storing open source intelligence
9. Why is it important to store system logs remotely?
- A. Local systems can't handle it.
 - B. Bandwidth is faster than disks.
 - C. Attackers might delete local logs.
 - D. It will defend against attacks.
10. What would be necessary for a TCP conversation to be considered established by a stateful firewall?
- A. Final acknowledgment message
 - B. Three-way handshake complete

- C. Sequence numbers aligned
 - D. SYN message received
11. What is the purpose of a security policy?
- A. To provide high-level guidance on the role of security
 - B. To provide specific direction to security workers
 - C. To increase the bottom line of a company
 - D. To align standards and practices
12. What additional properties does the Parkerian hexad offer over the CIA triad?
- A. Confidentiality, awareness, authenticity
 - B. Utility, awareness, possession
 - C. Utility, possession, authenticity
 - D. Possession, control, authenticity
13. What important event can be exposed by enabling auditing?
- A. System shutdown
 - B. Service startup
 - C. Package installation
 - D. User login
14. What can an intrusion prevention system do that an intrusion detection system can't?
- A. Generate alerts
 - B. Block or reject network traffic
 - C. Complete the three-way handshake to bogus messages
 - D. Log packets
15. Which of these is an example of an application layer gateway?
- A. Web application firewall
 - B. Runtime application self-protection
 - C. Java applet
 - D. Intrusion prevention system
16. Which information would a packet filter use to make decisions about what traffic to allow into the network?
- A. HTTP REQUEST message
 - B. Ethernet type
 - C. UDP source port
 - D. SNMP OID
17. Which of the following products might be used as an intrusion detection system?
- A. Elastic Stack
 - B. Prewikka
 - C. Snort
 - D. Snorby
18. Which of these isn't an example of an attack that compromises integrity?
- A. Buffer overflow
 - B. Man in the middle
 - C. Heap spraying
 - D. Watering hole
19. What type of attack could lead to a direct compromise of availability?
- A. Watering hole
 - B. DoS
 - C. Phishing
 - D. Buffer overflow

20. What important function can EDR offer to security operations staff?
- A. Host isolation
 - B. Malware detection
 - C. Remote data collection
 - D. All of the above
21. Which type of security control is a firewall?
- A. Administrative
 - B. Physical
 - C. Technical
 - D. Corrective
22. Management has been informed of a risk to personally identifiable information (PII) that results from an application being developed and managed by the company. They have chosen not to do anything with the risk. What risk management approach have they taken?
- A. Risk transference
 - B. Risk avoidance
 - C. Risk mitigation
 - D. Risk acceptance
23. You've been asked to implement a set of standards to support a policy. What type of security control are you developing?
- A. Administrative
 - B. Corrective
 - C. Logical
 - D. Functional
24. Your risk management team has asked for a technical control that could mitigate the risk that may be associated with insider threat. Which of these controls would work for that?
- A. Security policy
 - B. Identity and access management (IAM) solution
 - C. Security standards
 - D. Host-based firewall
25. An attacker has registered the domain name facebookmailings.com which will be used to send phishing messages out. Which of the MITRE ATT&CK Framework categories would that fall into?
- A. Initial access
 - B. Lateral movement
 - C. Credential access
 - D. Resource development

Table of contents collapsed